DATABASE PROJECT 1

Affordable Housing Maintenance
Work-Order Database

Case 5 | CSE 5720 — Database Systems | Fall 2026

Deliverable 6: Project README and Demonstration Guide

1. Project Information

Item	Details
Case	Case 5 → Affordable Housing Maintenance Work Orders
Course	CSE 5720 → Database Systems
Database platform	PostgreSQL
Database type	Relational database using synthetic test data
Purpose	Organize maintenance work orders, technician assignments, status tracking, and management reporting for a fictional affordable housing nonprofit.

2. Project Overview

This project develops a relational database system for a fictional affordable housing nonprofit organization that will manage residential properties and maintenance requests. The database consists of a variety of information about properties, units, resident codes, issue categories, technicians, work orders, technician assignments as well as work-order status history and primarily focuses on relational database design, data integrity, SQL reporting and verification. That said, it doesn't include a website, mobile application, payment processing, real resident personal information, or a complete property-management system.

The system is implemented utilizing PostgreSQL and consists of 8 relational entities along with a view named work_order_details. The database utilizes primary keys, foreign keys, unique constraints, check constraints and more in order to ensure the data's integrity. It further includes 5 SQL queries designed to support staff in reviewing maintenance requests, identify overdue high-priority work orders, review technician assignments, as well as analyzing work orders by property and unit.

Table	Purpose
PROPERTY	Stores property information

UNIT	Stores housing units and associates each unit with a property
RESIDENT_CODE	Stores coded resident identifiers without real resident personal information
ISSUE_CATEGORY	Defines categories used to classify maintenance issues
TECHNICIAN	Stores technician identifiers and related information
WORK_ORDER	Records maintenance requests, priorities, current statuses, and timestamps
WORK_ASSIGNMENT	connects work orders with technicians and assignment information
WORK_STATUS_HISTORY	Records work-order status changes over time

3. Client Problem and Goals

The client's main problem is maintaining reliable records of maintenance requests and using those to determine which issues need attention. The database is designed to support three main decision areas:

Decision area	Database-supported question
Overdue maintenance	Which open, high-priority work orders are older than seven days and may require follow-up?
Technician workload	Which technicians are assigned at least four open work orders?
Recurring maintenance	How many work orders are associated with each property and issue category, and which units have more work orders than the average unit?

4. Database Design and ERD

The following conceptual ERD represents the eight entities listed in Section 2 along with their attributes and individual relationships.

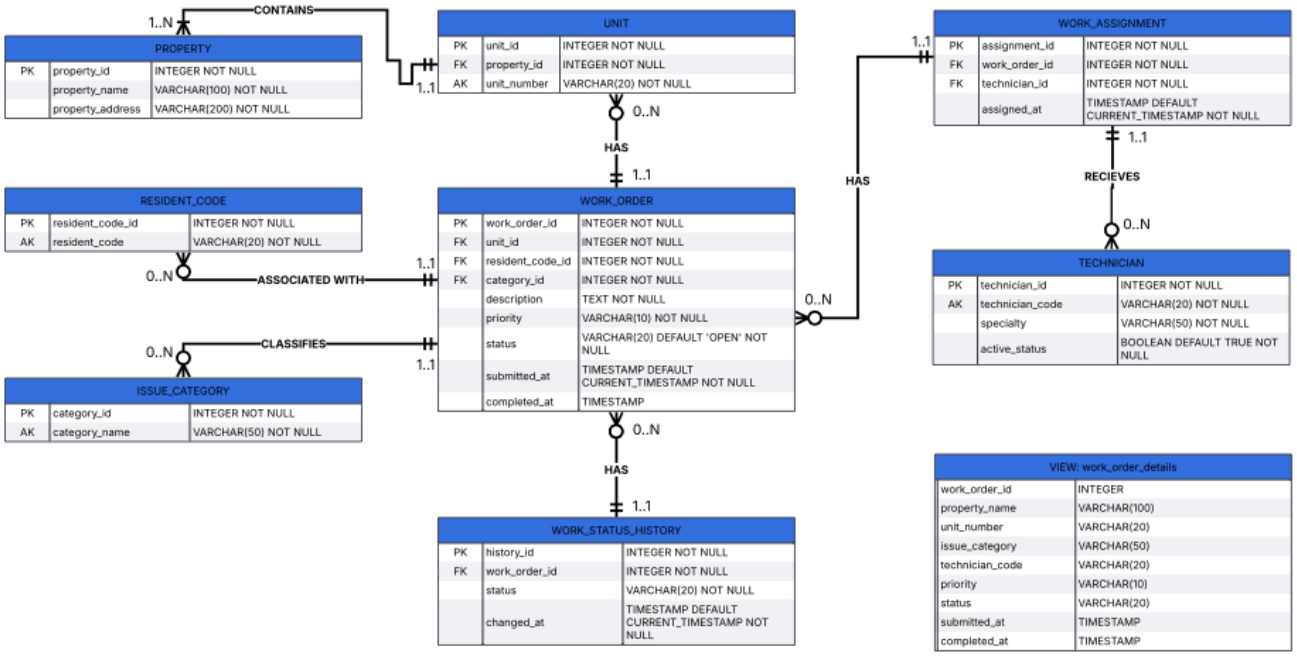


Figure 1. Conceptual ERD

4.1 Relationship Summary

Relationship	Description
PROPERTY → UNIT	Each property can contain zero or many units while each unit belongs to exactly one property. Unit numbers must be unique within a property, although different properties may use the same unit number
UNIT → WORK_ORDER	Each unit may have zero or many work orders while each work order concerns exactly one unit
RESIDENT_CODE → WORK_ORDER	Each resident code can be associated with zero or many work orders while each work order references exactly one coded resident record
ISSUE_CATEGORY → WORK_ORDER	Each issue category may classify zero or many work orders while each work order refers to exactly one existing issue category
WORK_ORDER → TECHNICIAN	Each work order may have 0 or many technicians assigned, while each technician may be assigned to 0 or many work orders. This many-to-many relationship is implemented through the associative entity WORK_ASSIGNMENT, which records the work order, technician, and assignment timestamp. The current implementation supports multiple technician assignments but does not track assignment removal or reassignment history
WORK_ORDER → WORK_STATUS_HISTORY	Each work order may have zero or many status-history entries where each history entry references one existing work order and records a status and timestamp.

The relational schema translates these entities and relationships into tables using primary keys, foreign keys, uniqueness rules, and integrity constraints.

5. PostgreSQL Setup and Execution Instructions

5.1 Requirements

The project utilizes PostgreSQL as its relational database management system. An SQL client, such as DBeaver or the PostgreSQL command-line interface (psql) is required to connect to the database and execute the project SQL scripts.

5.2 Database Setup

1. Open DBeaver and connect to the PostgreSQL server
2. Create a new and empty database for the project
3. Connect to newly created database
4. Open the four project SQL files and connect each to existing database
5. Execute the scripts in the following order:
   - 01_schema.sql – Creates the eight relational tables, their PKs, FKs, constraints, along with the work_order_details view
   - 02_data.sql – Inserts the synthetic sample data used to demonstrate the affordable housing maintenance scenario
   - 03_queries.sql – Executes the 5 queries that address the client's operational and management questions
   - 04_verification.sql – performs database integrity checks and verifies constraints
6. Review the query results and verification outputs and confirm that the database is functioning as intended

5.3 SQL File Execution Order

Order / file	Purpose
1. 01_schema.sql	Creates the 8 database tables, PKs, FKs, unique constraints, check constraints, along with the work_order_details view
2. 02_data.sql	Inserts synthetic records representing properties, residential units, resident codes, issue categories, technicians, work orders, assignments, and work-order status history
3. 03_queries.sql	Contains and executes five labeled SQL queries, addressing the client's operational and management questions.
4. 04_verification.sql	Tests database constraints and different scenarios.

	The script includes intentional invalid data tests that utilize transaction rollback to prevent test records from remaining in the database. Also includes queries that cover unassigned work orders, status and completion timestamp consistency, orphan status-history records, and work-order summaries

5.4 Expected Execution Results

· After successfully executing 01_schema.sql and 02_data.sql, the database should contain the eight relational tables, the work_order_details view, and the synthetic data.

· The 03_queries.sql script should execute all five reporting queries and produce results addressing the project's required client questions.

· The 04_verification.sql script demonstrates that the selected integrity constraints reject invalid data and further provides additional outputs for reviewing database relationships and work-order information. The intentional invalid-data tests should be rolled back so that unsuccessful test records do not remain in the database.

6. Query Portfolio and Expected Outputs

The following queries support the client's operational and management decisions.

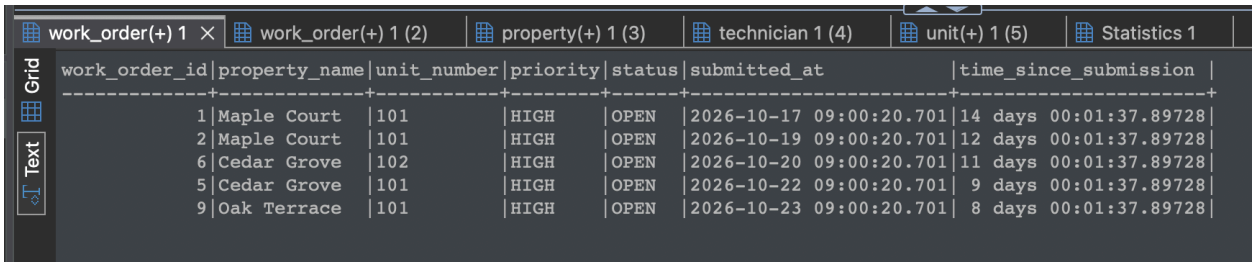
Q1. Overdue High-Priority Work Orders

Client question: List open high-priority work orders older than 7 days

Final SQL query:

```
SELECT
  wo.work_order_id,
  p.property_name,
  u.unit_number,
  wo.priority,
  wo.status,
  wo.submitted_at,
  CURRENT_TIMESTAMP - wo.submitted_at AS time_since_submission
FROM WORK_ORDER AS wo
JOIN UNIT AS u
  ON wo.unit_id = u.unit_id
JOIN PROPERTY AS p
  ON u.property_id = p.property_id
WHERE wo.status = 'OPEN'
 AND wo.priority = 'HIGH'
 AND wo.submitted_at < CURRENT_TIMESTAMP - INTERVAL '7 days'
ORDER BY wo.submitted_at ASC;
```

Output / result:



work_order_id	property_name	unit_number	priority	status	submitted_at	time_since_submission
1	Maple Court	101	HIGH	OPEN	2026-10-17 09:00:20.701	14 days 00:01:37.89728
2	Maple Court	101	HIGH	OPEN	2026-10-19 09:00:20.701	12 days 00:01:37.89728
6	Cedar Grove	102	HIGH	OPEN	2026-10-20 09:00:20.701	11 days 00:01:37.89728
5	Cedar Grove	101	HIGH	OPEN	2026-10-22 09:00:20.701	9 days 00:01:37.89728
9	Oak Terrace	101	HIGH	OPEN	2026-10-23 09:00:20.701	8 days 00:01:37.89728

Result: 5 qualifying work orders: 1, 2, 5, 6, 9

Explanation:

This query identifies open, high-priority work orders that are older than seven days. It displays the associated property_name and unit_number, along with the priority, current status, submission timestamp, and time since submission. The results are ordered by submission date, with the oldest work orders appearing first, this ultimately helps management identify requests that need the most attention.

Q2. Work-Order Details

Client question: Show each work order with its property, unit, issue category, assigned technician, and status

Final SQL query:

```sql
SELECT
  wo.work_order_id,
  p.property_name,
  u.unit_number,
  ic.category_name AS issue_category,
  t.technician_code,
  wo.priority,
  wo.status,
  wo.submitted_at,
  wo.completed_at
FROM WORK_ORDER AS wo
JOIN UNIT AS u
  ON wo.unit_id = u.unit_id
JOIN PROPERTY AS p
  ON u.property_id = p.property_id
JOIN ISSUE_CATEGORY AS ic
  ON wo.category_id = ic.category_id
LEFT JOIN WORK_ASSIGNMENT AS wa
  ON wo.work_order_id = wa.work_order_id
LEFT JOIN TECHNICIAN AS t
  ON wa.technician_id = t.technician_id
ORDER BY wo.work_order_id, t.technician_code;
```

Output / result:

work_order(+) 1 | work_order(+) 1 (2) | property(+) 1 (3) | technician 1 (4) | unit(+) 1 (5) | Statistics 1

Enter a SQL expression to filter results (use Ctrl+Space)

work_order_id	property_name	unit_number	issue_category	technician_code	priority	status	submitted_at	completed_at
1	Maple Court	101	Plumbing	TECH-001	HIGH	OPEN	2026-10-17 09:00:20.701	
1	Maple Court	101	Plumbing	TECH-002	HIGH	OPEN	2026-10-17 09:00:20.701	
2	Maple Court	101	Plumbing	TECH-001	HIGH	OPEN	2026-10-19 09:00:20.701	
3	Maple Court	102	Plumbing	TECH-001	MEDIUM	OPEN	2026-10-21 09:00:20.701	
4	Maple Court	103	Plumbing	TECH-001	LOW	OPEN	2026-10-25 09:00:20.701	
5	Cedar Grove	101	Plumbing	TECH-001	HIGH	OPEN	2026-10-22 09:00:20.701	
6	Cedar Grove	102	Electrical	TECH-002	HIGH	OPEN	2026-10-20 09:00:20.701	
6	Cedar Grove	102	Electrical	TECH-003	HIGH	OPEN	2026-10-20 09:00:20.701	
7	Cedar Grove	103	Doors and Locks	TECH-002	MEDIUM	OPEN	2026-10-26 09:00:20.701	
8	Cedar Grove	104	Appliance	TECH-002	HIGH	OPEN	2026-10-28 09:00:20.701	
9	Oak Terrace	101	Heating and Cooling	TECH-002	HIGH	OPEN	2026-10-23 09:00:20.701	
10	Oak Terrace	102	Pest Control		LOW	OPEN	2026-10-29 09:00:20.701	
11	Maple Court	101	Electrical	TECH-002	MEDIUM	IN_PROGRESS	2026-10-24 09:00:20.701	
11	Maple Court	101	Electrical	TECH-003	MEDIUM	IN_PROGRESS	2026-10-24 09:00:20.701	
12	Maple Court	102	Appliance	TECH-002	MEDIUM	IN_PROGRESS	2026-10-27 09:00:20.701	
13	Cedar Grove	101	Heating and Cooling	TECH-004	HIGH	IN_PROGRESS	2026-10-25 09:00:20.701	
14	Oak Terrace	101	Doors and Locks	TECH-002	LOW	IN_PROGRESS	2026-10-28 09:00:20.701	
15	Maple Court	101	Plumbing	TECH-001	MEDIUM	COMPLETED	2026-10-01 09:00:20.701	2026-10-06 09:00:20.701
16	Maple Court	102	Electrical	TECH-003	MEDIUM	COMPLETED	2026-10-11 09:00:20.701	2026-10-13 09:00:20.701
17	Cedar Grove	101	Plumbing	TECH-001	HIGH	COMPLETED	2026-10-13 09:00:20.701	2026-10-16 09:00:20.701
18	Cedar Grove	102	Appliance	TECH-002	LOW	COMPLETED	2026-10-19 09:00:20.701	2026-10-21 09:00:20.701
19	Maple Court	103	Doors and Locks		LOW	CANCELLED	2026-10-26 09:00:20.701	
20	Oak Terrace	102	Pest Control		LOW	CANCELLED	2026-10-27 09:00:20.701	

Result: all 20 work orders, represented by 23 rows because some have multiple technicians.

Explanation:

This query provides a detailed overview of maintenance work orders by displaying their associated properties, units, issue categories, assigned technicians, priorities, current status, time of submission, as well as time of completion. Furthermore, the use of LEFT JOIN preserves work orders without an assigned technician.

Q3. Work Orders by Property and Issue Category

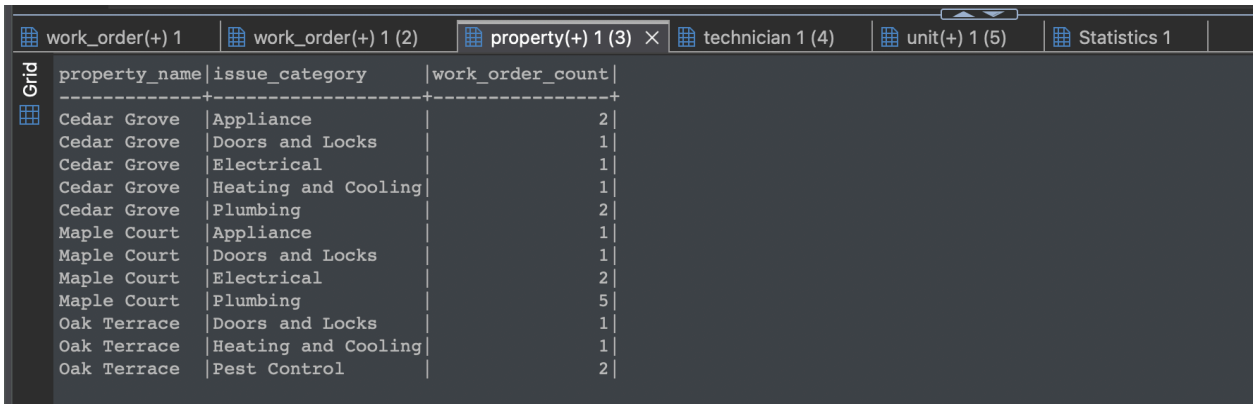
Client question: Count work orders by property and issue category

Final SQL query:

```sql
SELECT
  p.property_name,
  ic.category_name AS issue_category,
  COUNT(wo.work_order_id) AS work_order_count
FROM WORK_ORDER AS wo
JOIN UNIT AS u
  ON wo.unit_id = u.unit_id
JOIN PROPERTY AS p
  ON u.property_id = p.property_id
JOIN ISSUE_CATEGORY AS ic
  ON wo.category_id = ic.category_id
GROUP BY
  p.property_id,
  p.property_name,
  ic.category_id,
```

```
  ic.category_name
ORDER BY
  p.property_name,
  ic.category_name;
```

Output / result:



property_name	issue_category	work_order_count
Cedar Grove	Appliance	2
Cedar Grove	Doors and Locks	1
Cedar Grove	Electrical	1
Cedar Grove	Heating and Cooling	1
Cedar Grove	Plumbing	2
Maple Court	Appliance	1
Maple Court	Doors and Locks	1
Maple Court	Electrical	2
Maple Court	Plumbing	5
Oak Terrace	Doors and Locks	1
Oak Terrace	Heating and Cooling	1
Oak Terrace	Pest Control	2

Result: 12 property / issue category groupings with number of work orders.

Explanation: This query summarizes maintenance activity by counting the number of work orders associated with each property and issue category, and by grouping the results by property and category, it allows management to compare maintenance activity across properties and determine the types of issues being most reported.

Q4. Technicians with at Least Four Open Work Orders

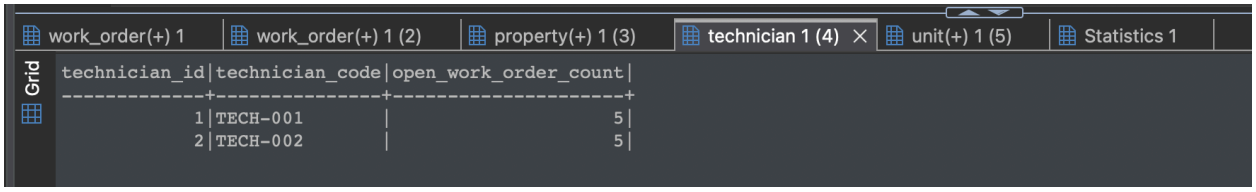
Client question: Show technicians assigned at least 4 open work orders

Final SQL query:

```
SELECT
  t.technician_id,
  t.technician_code,
  COUNT(DISTINCT wo.work_order_id) AS open_work_order_count
FROM TECHNICIAN AS t
JOIN WORK_ASSIGNMENT AS wa
  ON t.technician_id = wa.technician_id
JOIN WORK_ORDER AS wo
  ON wa.work_order_id = wo.work_order_id
WHERE wo.status = 'OPEN'
GROUP BY
  t.technician_id,
  t.technician_code
HAVING COUNT(DISTINCT wo.work_order_id) >= 4
```

```
ORDER BY open_work_order_count DESC, t.technician_code;
```

Output / result:



Result: TECH-001 & TECH-002 are both assigned 5 work orders

Explanation: This query identifies technicians who have been assigned at least 4 open work orders. It counts each distinct work order assigned to a technician and displays the technician's information alongside the total count. This in particular helps management review technician workloads and identify employees who have several assignments.

Q5. Units with Above-Average Work Orders

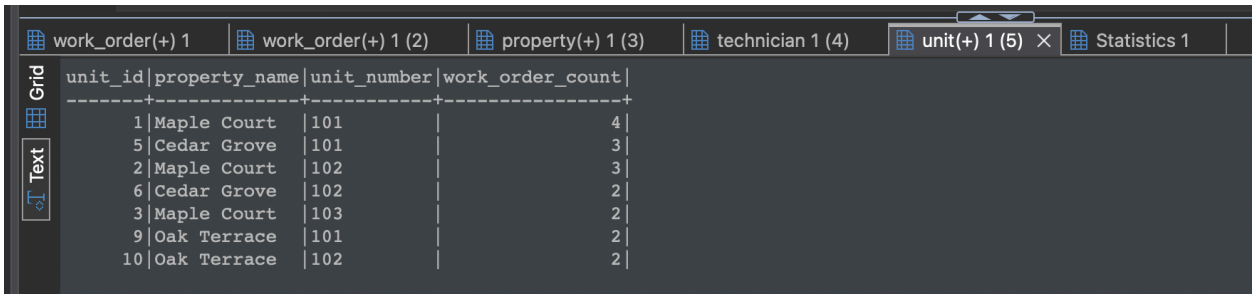
Client question: Find units with more work orders than the average unit, using a nested query or CTE

Final SQL query:

```
WITH unit_work_order_counts AS (
  SELECT
    u.unit_id,
    p.property_name,
    u.unit_number,
    COUNT(wo.work_order_id) AS work_order_count
  FROM UNIT AS u
  JOIN PROPERTY AS p
    ON u.property_id = p.property_id
  LEFT JOIN WORK_ORDER AS wo
    ON u.unit_id = wo.unit_id
  GROUP BY
    u.unit_id,
    p.property_name,
    u.unit_number
)
SELECT
  unit_id,
  property_name,
  unit_number,
  work_order_count
FROM unit_work_order_counts
```

```
WHERE work_order_count > (
  SELECT AVG(work_order_count::NUMERIC)
  FROM unit_work_order_counts
)
ORDER BY work_order_count DESC, property_name, unit_number;
```

Output / result:



unit_id	property_name	unit_number	work_order_count
1	Maple Court	101	4
5	Cedar Grove	101	3
2	Maple Court	102	3
6	Cedar Grove	102	2
3	Maple Court	103	2
9	Oak Terrace	101	2
10	Oak Terrace	102	2

Result: 7 units with above-average work-order counts

Explanation: This query identifies residential units with more work orders than the average number of work orders across all units. The CTE calculates the work-order count for each unit, including units with no work orders seeing as how a LEFT JOIN was used. A nested query then calculates the average, allowing the final results to highlight units with above-average maintenance activity.

7. Verification and Integrity Testing

The database was tested using the 04_verification.sql script to confirm that the implemented tables, relationships, and integrity constraints worked as required. The verification process includes 10 tests covering initial table row counts, constraint enforcement, unassigned work orders, work-order timestamp consistency, orphan status-history records, and summary counts by status and priority.

5 tests specifically evaluated whether PostgreSQL rejects invalid data, these tests check duplicate unit numbers within the same property, invalid priority values, invalid work-order statuses, completion timestamps earlier than submission timestamps, and work orders referencing nonexistent units. Each of these five tests produced a PASS result, confirming that the corresponding UNIQUE, CHECK, and FOREIGN KEY constraints were enforced.

The remaining verification queries analyze the existing sample data and database relationships, specifically they report the number of records in each table, identify work orders without technician assignments, check for inconsistencies between work-order statuses and completion timestamps, identify orphan status-history records, and summarize work orders by status and priority.

Screenshots of the execution results are included below as evidence of the verification process. The complete SQL statements are provided in the accompanying 04_verification.sql file.

Constraint Tests

Test	Expected behavior	Observed result
Duplicate unit number 	Reject duplicate unit numbers within the same property	PASS – Duplicate unit number was rejected.
Invalid priority	Reject priority values outside LOW, MEDIUM, HIGH	PASS – Invalid priority was rejected.

```
BEGIN;

DO $$
DECLARE
    v_property_id INTEGER;
    v_unit_number VARCHAR(20);
    v_unit_id INTEGER;
    v_rejected BOOLEAN := FALSE;
BEGIN
    SELECT property_id, unit_number
    INTO v_property_id, v_unit_number
    FROM UNIT
    ORDER BY unit_id
    LIMIT 1;

    IF v_property_id IS NULL THEN
        RAISE WARNING 'FAIL: TEST 2 skipped; UNIT has no rows.';
    ELSE
        SELECT COALESCE(MAX(unit_id), 0) + 100000
        INTO v_unit_id
        FROM UNIT;

        BEGIN
            INSERT INTO UNIT (unit_id, property_id, unit_number)
            VALUES (v_unit_id, v_property_id, v_unit_number);
        EXCEPTION WHEN unique_violation THEN
            v_rejected := TRUE;
        END;

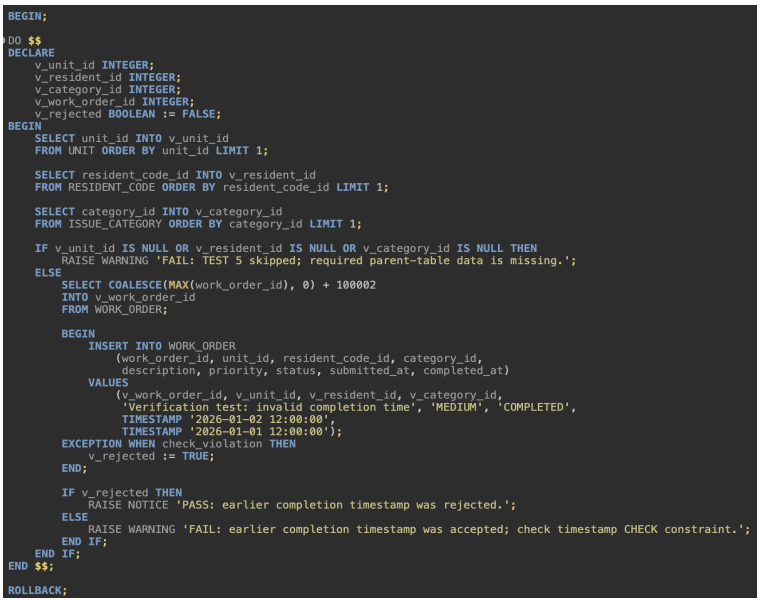
        IF v_rejected THEN
            RAISE NOTICE 'PASS: duplicate unit number was rejected.';
        ELSE
            RAISE WARNING 'FAIL: duplicate unit number was accepted; check UNIQUE(property_id, unit_number).';
        END IF;
    END IF;
END $$;

ROLLBACK;
```

Constraint Tests

		
Invalid Status 	Reject status values outside the permitted options	PASS – Invalid status was rejected.
Completion timestamp	Reject a completion timestamp earlier than submission	PASS – Earlier completion timestamp was rejected

Constraint Tests

		
Foreign Key 	Reject a work order Referencing a nonexistent unit	PASS – Nonexistent unit_id was rejected

The verification script was executed successfully in PostgreSQL and the 5 constraint tests confirmed that the database rejects duplicate unit numbers within the same property, invalid priority and status values, completion timestamps earlier than submission, and work orders referencing nonexistent units. The results thereby demonstrate that the implemented UNIQUE, CHECK, and FOREIGN KEY constraints enforce the selected data-integrity rules as intended.

Additional Checks

Check	Expected result based on synthetic data	Output / result

Table row counts
`SELECT 'PROPERTY' AS table_name, COUNT(*) AS row_count FROM PROPERTY`
`UNION ALL SELECT 'UNIT', COUNT(*) FROM UNIT`
`UNION ALL SELECT 'RESIDENT_CODE', COUNT(*) FROM RESIDENT_CODE`
`UNION ALL SELECT 'ISSUE_CATEGORY', COUNT(*) FROM ISSUE_CATEGORY`
`UNION ALL SELECT 'TECHNICIAN', COUNT(*) FROM TECHNICIAN`
`UNION ALL SELECT 'WORK_ORDER', COUNT(*) FROM WORK_ORDER`
`UNION ALL SELECT 'WORK_ASSIGNMENT', COUNT(*) FROM WORK_ASSIGNMENT`
`UNION ALL SELECT 'WORK_STATUS_HISTORY', COUNT(*) FROM WORK_STATUS_HISTORY`
`ORDER BY table_name;`
	3 properties,
	12 units,
	10 resident codes,
	6 categories,
	5 technicians,
	20 work orders,
	20 assignments,
	30 history records
		table_name | row_count
		ISSUE_CATEGORY | 6
		PROPERTY | 3
		RESIDENT_CODE | 10
		TECHNICIAN | 5
		UNIT | 12
		WORK_ASSIGNMENT | 20
		WORK_ORDER | 20
		WORK_STATUS_HISTORY | 30

Unassigned
`SELECT`
`wo.work_order_id,`
`wo.status,`
`wo.priority,`
`wo.submitted_at`
`FROM WORK_ORDER AS wo`
`LEFT JOIN WORK_ASSIGNMENT AS wa`
`ON wo.work_order_id = wa.work_order_id`
`WHERE wa.assignment_id IS NULL`
`ORDER BY wo.submitted_at;`
	Work orders 19 and 20	work_order_id | status | priority | submitted_at
		19 | CANCELLED | LOW | 2026-10-26 09:00:20.701
		20 | CANCELLED | LOW | 2026-10-27 09:00:20.701
		10 | OPEN | LOW | 2026-10-29 09:00:20.701

Status/Completion
`SELECT`
`work_order_id,`
`status,`
`submitted_at,`
`completed_at`
`FROM WORK_ORDER`
`WHERE (status = 'COMPLETED' AND completed_at IS NULL)`
`OR (status <> 'COMPLETED' AND completed_at IS NOT NULL)`
`ORDER BY work_order_id;`
	No records	work_order_id | status | submitted_at | completed_at

Orphan status-history records
`SELECT`
`h.history_id,`
`h.work_order_id,`
`h.status,`
`h.changed_at`
`FROM WORK_STATUS_HISTORY AS h`
`LEFT JOIN WORK_ORDER AS wo`
`ON h.work_order_id = wo.work_order_id`
`WHERE wo.work_order_id IS NULL;`
	No records	history_id | work_order_id | status | changed_at

Additional Checks

Work-order summary

```sql
-- Work orders grouped by status.
SELECT
    status,
    COUNT(*) AS work_order_count
FROM WORK_ORDER
GROUP BY status
ORDER BY status;

-- Work orders grouped by priority.
SELECT
    priority,
    COUNT(*) AS work_order_count
FROM WORK_ORDER
GROUP BY priority
ORDER BY priority;
```

2 cancelled, 4 completed, 4 in progress, 10 open

```
status     |work_order_count|
-----------+----------------+
CANCELLED  |               2|
COMPLETED  |               4|
IN_PROGRESS|               4|
OPEN       |              10|
```

```
priority|work_order_count|
--------+----------------+
HIGH    |               8|
LOW     |               6|
MEDIUM  |               6|
```

8. How the Database Helps the Client

The affordable housing nonprofit organization in Case 5 is in need of a reliable way to organize maintenance requests across properties and residential units. This project addresses that need by constructing a relational database that records maintenance requests, technician assignments, and work-order information.

The database supports the nonprofit's maintenance workflow by providing staff a way to connect each work order with an existing residential unit, resident code, and issue category. In addition, work orders can be assigned to technicians when necessary, while status history records create a way to document any changes to the work order. By connecting these records through primary keys and foreign keys, the database helps maintain consistent/reliable relationships/individual information from each entity.

Furthermore, the 5 SQL queries provide managers with information that help in reviewing maintenance requests and making decisions. The first query identifies open, high-priority work orders that are more than 7 days old, helping staff identify requests that may require follow-up. The second query provides an overview of work orders, including their associated properties, units, issue categories, technicians, and current statuses. The third query counts work orders by property and issue category, allowing managers to examine maintenance activity across different locations and types of issues. The fourth query identifies technicians assigned at least four open work orders, providing information that can help managers review technician workloads. Finally, the fifth query identifies units with more work orders than the average unit, helping managers recognize locations with higher maintenance requests.

Even more, database integrity is yet another important part of the project. Primary keys uniquely identify records, foreign keys maintain relationships between tables, and constraints restrict invalid values. For instance, priority and status values are limited to permitted options, duplicate unit numbers within the same property are rejected, and completion timestamps cannot precede submission timestamps. The verification script in particular provides additional evidence that these selected integrity rules are enforced by PostgreSQL.

All in all, the database provides the nonprofit with a structured way to record, organize, review, and analyze maintenance work orders, and while the project doesn’t include a website, mobile application, payment processing, real resident personal information, or a complete property-management system, it certainly demonstrates how a relational database can reliably help in efficiently keeping records and reporting.

9. Team Contributions and Roles

List team member's confirmed responsibilities for database design, SQL implementation, testing, documentation, and demonstration preparation.

Team member	Responsibilities / contributions
Harshil Patel	Completed 03_queries.sql & 04_verification.sql for Deliverable 5, including the five required queries and database integrity tests.
Giovanni De La Torre	Assisted with finalizing Deliverable 6 by reviewing the documentation, SQL results, expected outputs, verification results, and demonstration plan. Coordinated the team's recording sections and helped prepare and edit the final project demonstration video.
Daniel Santoscoy	Completed 01_schema.sql & 02_data.sql for Deliverable 4, including the database tables, primary keys, foreign keys, constraints, relationships, and synthetic test data.
Edgar Hernandez	Completed Deliverables 1-3, including the client scope, business rules, scenarios, ERD, relational schema, and data dictionary. Prepared Deliverable 6, including the README, setup instructions, query explanations, verification section, and demonstration outline.

10. Demonstration Preparation

The team will record individual presentation sections and combine them into one final 6-8 minute MP4 demonstration. Each member will show the relevant ERD, SQL, file, documentation, or query results while explaining their assigned section.

1. Project Introduction - Edgar Hernandez

   Approximate time: 1 minute. Edgar will introduce the affordable housing nonprofit, explain the maintenance-tracking problem, identify the primary users, and describe the purpose of the database.

2. Database Design - Edgar Hernandez

   Approximate time: 1 minute. Edgar will present the finalized ERD and explain the eight tables, their main attributes, and their relationships.

3. SQL Implementation - Daniel Santoscoy

   Approximate time: 1 minute. Daniel will explain 01_schema.sql and 02_data.sql, including the tables creation order, primary keys, foreign keys, constraints, and synthetic test data.

4. Query Demonstrations - Harshil Patel

   Approximate time: 2 minutes. Harshil will demonstrate selected queries from 03_queries.sql, including Q1, an aggregate query such as Q3 or Q4 and Q5 using a nested query or CTE. He will explain what the results show and how they support the client's decisions.

5. Verification and Integrity Testing - Harshil Patel

   Approximate time: 1 minute. Harshil will demonstrate an invalid data entry being rejected by PostgreSQL, such as an invalid completion timestamp, invalid status, or invalid foreign key. He will explain how 04_verification.sql uses transactions and ROLLBACK to safely test the database.

6. Design Decisions and Lessons Learned - Giovanni De La Torre

Approximate time: 1 minute. Giovanni will discuss important design decisions, including the use of primary keys, foreign keys, check constraints, the handling of unassigned work orders, and the challenges the team encountered during development.

7. Client Benefit and Conclusion - Giovanni De La Torre

Approximate time: 1 minute. Giovanni will explain how the database helps staff record maintenance requests, identify overdue work, review technician workloads, and recognize recurring maintenance problems. He will conclude the demonstration by summarizing how the database addresses the nonprofit's needs.